

Test Report: DynamicArray & Stack

src/tests/DataStructureTests.java — Data-Structures-Branch

Six required tests, one resize trace table, and per-structure explanations

TECH INTELLECTUALS · Campus Service Hub

Table of Contents

1. Compilation Status — Read This First
2. DynamicArray: How It Works
3. DynamicArray: Resize Trace Table
4. DynamicArray: Test Methods
5. Stack: How It Works
6. Stack: Known Gap (get/set)
7. Stack: Test Methods
8. Expected Test Behavior
9. Appendix: Full Test Code

1. Compilation Status — Read This First

Blocker: Stack.java will not compile as-is

Stack<T> declares “implements DataStructure<T>”, and that interface requires get(int index) and set(int index, T item). The version of Stack.java shared for this task does not include either method. Until they are added (or the class is changed to not implement DataStructure<T>), the file fails to compile — which means none of the tests below, including the ones for DynamicArray, can run as part of the same test class until this is resolved.

This report was prepared without a working Java compiler available in the environment used to write it (a JDK could not be installed, and the required package servers were unreachable). Because of that, the “Expected Test Behavior” section further down is a manual trace through the code's logic, not console output from an actual run. Before trusting this as final proof the tests pass, run them locally and compare the real output against what is predicted here.

2. DynamicArray: How It Works

A DynamicArray is a list backed by a plain array that automatically grows when it runs out of room. Internally it tracks a count (how many slots are actually used) separately from the backing array's true length (its capacity), so it can tell the difference between “full” and “has room.” When `add()` is called and the array is full, `growArray()` builds a new array at double the capacity and copies every existing element into it — an $O(n)$ operation — before the new item is placed.

Because doubling happens less and less frequently as the array grows, the average cost of `add()` across many calls works out to $O(1)$, even though any individual call might occasionally trigger a full copy; this is what's meant by “amortized constant time.” `get()` and `set()` remain simple $O(1)$ direct-index lookups throughout, since they never need to search — they read or write a specific array position directly.

2.1 Relevant source

```
public void add(T item) {
    if (count == data.length) {
        growArray();
    }
    data[count] = item;
    count = count + 1;
}

private void growArray() {
    Object[] biggerArray = new Object[data.length * 2];
    for (int i = 0; i < data.length; i++) {
        biggerArray[i] = data[i];
    }
    data = biggerArray;
}
```

3. DynamicArray: Resize Trace Table

DynamicArray starts with a backing array of capacity 10. Each time `add()` is called and `count == data.length`, `growArray()` doubles the capacity and copies every existing element across. The trace below inserts integers 0 through 24 (25 items total) one at a time, recording the array's state immediately before and immediately after each point where a resize is triggered.

Step (insert value)	count before add	capacity before add	Resize triggered?	capacity after add
insert 0	0	10	no	10
insert 1–8	1–8	10	no	10
insert 9	9	10	no	10
insert 10	10	10	YES — array full	20
insert 11–19	11–19	20	no	20
insert 20	20	20	YES — array full	40
insert 21–24	21–24	40	no	40

3.1 Full contents at each resize point

Point	Contents
Before insert 10 (full at capacity 10)	[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
After insert 10 (resized to capacity 20)	[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Before insert 20 (full at capacity 20)	[0, 1, 2, ... 19]
After insert 20 (resized to capacity 40)	[0, 1, 2, ... 20]
Final state after 25 inserts	count = 25, capacity = 40, contents [0 ... 24]

What this shows

Capacity grows $10 \rightarrow 20 \rightarrow 40$, doubling exactly at the two points where the array was completely full right before an `add()` call. Capacity (40) ends up larger than what's actually used (25 elements) — the extra 15 slots are unused headroom, reserved so the next 15 inserts won't need another resize. This is the space/time trade-off behind amortized $O(1)$ `add()`: occasionally overshooting on memory so most calls don't pay the $O(n)$ cost of copying the whole array.

4. DynamicArray: Test Methods

Three required tests, following the existing `testBinarySearchTree()` pattern in the file.

4.1 Normal case

```
private void testDynamicArrayNormal() {
    DynamicArray<Integer> array = new DynamicArray<>();
    array.add(10);
    array.add(20);
    array.add(30);
    this.assertEquals(3, array.size(), "DynamicArray size after inserting 3 items");
    this.assertEquals(10, array.get(0), "DynamicArray value at index 0");
    this.assertEquals(20, array.get(1), "DynamicArray value at index 1");
    this.assertEquals(30, array.get(2), "DynamicArray value at index 2");
}
```

4.2 Boundary case

```
private void testDynamicArrayBoundary() {
    DynamicArray<Integer> array = new DynamicArray<>();
    this.assertTrue(array.isEmpty(), "DynamicArray should start empty before any insert");

    array.add(42);
    this.assertTrue(!array.isEmpty(), "DynamicArray should not be empty after one insert");
    this.assertEquals(1, array.size(), "DynamicArray size should be 1 after a single insert");
    this.assertEquals(42, array.get(0), "DynamicArray single element should be retrievable at index 0");
}
```

4.3 Invalid case

```
private void testDynamicArrayInvalid() {
    DynamicArray<Integer> array = new DynamicArray<>();
    boolean threw = false;
    try {
        array.get(0);
    } catch (IndexOutOfBoundsException e) {
        threw = true;
    }
    this.assertTrue(threw, "get() on empty DynamicArray should throw IndexOutOfBoundsException");
}
```


5. Stack: How It Works

This Stack is not built directly on a raw array like `DynamicArray` — instead it wraps a custom `LinkedList<T>`, using `addFirst()` for `push()` and `removeAt(0)/getFirst()` for `pop()/peek()`. Treating the front of the linked list as the top of the stack means both operations run in $O(1)$, since a linked list can add or remove its first node without shifting anything else, unlike an array-backed structure where removing from the front would require shifting every remaining element.

The class enforces Last-In-First-Out (LIFO) ordering strictly: `pop()` always returns the most recently pushed item first. It also guards two edge cases deliberately: pushing null throws `IllegalArgumentException` immediately rather than silently storing an unusable entry, and calling `pop()` or `peek()` on an empty stack throws `IllegalStateException` rather than returning null or crashing with a less meaningful error.

5.1 Relevant source

```
public void push(T item) {
    if (item == null) {
        throw new IllegalArgumentException("Null elements not allowed");
    }
    items.addFirst(item);
}

public T pop() {
    if (isEmpty()) {
        throw new IllegalStateException("Stack is empty");
    }
    return items.removeAt(0);
}
```


6. Stack: Known Gap (get/set)

Interface contract not fully satisfied

`Stack<T>` declares `implements DataStructure<T>`, which requires `get(int index)` and `set(int index, T item)` per the interface contract, but neither method appears in the source as provided. Every other class in the project (`DynamicArray`, `DisjointSet`, `Graph`, and the existing `BST/BTree/CircularQueue/Deque` classes) satisfies the full interface, so `Stack` is currently the outlier.

This was not addressed as part of this testing task because index-based access has ambiguous semantics for a stack — does index 0 mean the top, the bottom, or a position in insertion order? That is a design decision for whoever owns this file, not something to guess silently in a test suite. No `get()/set()` tests were written as a result; the three tests in this report cover only the methods the class actually implements: `push`, `pop`, `peek`, `size`, and `isEmpty`.

7. Stack: Test Methods

7.1 Normal case

```
private void testStackNormal() {
    Stack<Integer> stack = new Stack<>();
    stack.push(1);
    stack.push(2);
    stack.push(3);
    this.assertEquals(3, stack.size(), "Stack size after 3 pushes");
    this.assertEquals(3, stack.peek(), "Stack peek should return the most recently
pushed item");
    this.assertEquals(3, stack.pop(), "Stack pop should return items in LIFO
order");
    this.assertEquals(2, stack.pop(), "Stack pop should return the second-to-last
pushed item next");
    this.assertEquals(1, stack.size(), "Stack size after two pops should reflect one
remaining item");
}
```

7.2 Boundary case

```
private void testStackBoundary() {
    Stack<Integer> stack = new Stack<>();
    this.assertTrue(stack.isEmpty(), "Stack should start empty before any push");

    stack.push(99);
    this.assertTrue(!stack.isEmpty(), "Stack should not be empty after one push");
    this.assertEquals(1, stack.size(), "Stack size should be 1 after a single
push");
    this.assertEquals(99, stack.peek(), "Stack peek should return the single pushed
item");

    this.assertEquals(99, stack.pop(), "Stack pop should return the single item");
    this.assertTrue(stack.isEmpty(), "Stack should be empty again after popping its
only item");
}
```

7.3 Invalid case

```
private void testStackInvalid() {
    Stack<Integer> stack = new Stack<>();
    boolean threwOnPop = false;
    try {
        stack.pop();
    }
```

```
    } catch (IllegalStateException e) {
        threwOnPop = true;
    }
    this.assertTrue(threwOnPop, "pop() on empty Stack should throw
IllegalStateException");

    boolean threwOnPeek = false;
    try {
        stack.peek();
    } catch (IllegalStateException e) {
        threwOnPeek = true;
    }
    this.assertTrue(threwOnPeek, "peek() on empty Stack should throw
IllegalStateException");
}
```

8. Expected Test Behavior

This is a manual trace, not a real console log

No JDK was available in the environment used to prepare this report, so the table below was produced by reading each method's logic step by step, not by compiling and running the code. Run the suite locally (after adding get()/set() to Stack.java) and compare against this table; if anything differs, trust your real output over this prediction.

Test method	Predicted result	Reasoning
testDynamicArrayNormal	PASS	add()/get() are straightforward $O(1)$ operations on 3 items, well under the resize threshold of 10.
testDynamicArrayBoundary	PASS	isEmpty() correctly reflects count == 0 before and after the single add().
testDynamicArrayInvalid	PASS	get(0) on count == 0 fails the $0 \leq \text{index} < \text{count}$ check and throws as expected.
testStackNormal	PASS	LinkedList.addFirst()/removeAt(0) correctly implement LIFO order for push/pop.
testStackBoundary	PASS	Single push/pop cycle correctly returns the stack to isEmpty() == true.
testStackInvalid	PASS	isEmpty() check in pop()/peek() triggers IllegalStateException on a fresh stack.

9. Appendix: Full Test Code Block

The six methods below, plus the two import lines and six calls into `runTests()`, are everything needed to add this coverage to the existing `src/tests/DataStructureTests.java` file.

```
import datastructures.DynamicArray;
import datastructures.Stack;

// ... inside the DataStructureTests class ...

public void runTests() {
    this.testBinarySearchTree();
    this.testBTree();
    this.testCircularQueue();
    this.testDeque();
    this.testDynamicArrayNormal();
    this.testDynamicArrayBoundary();
    this.testDynamicArrayInvalid();
    this.testStackNormal();
    this.testStackBoundary();
    this.testStackInvalid();
    System.out.println("Data structure tests passed.");
}
```